A

AapatVani AI

आपतवाणी · "the voice of disaster"

Community disaster reports — Nepali or English, text, photo or voice — turned into structured, traceable incident records, with an analysis of what a blocked road means for the communities that depend on it. Every claim carries its provenance, and a person confirms every fact.

SUBMITTED TO
Google Cloud AI Builder Cup 2026

THEME
Sustainability & Social Impact

CATEGORY
Social Good

LIVE PROTOTYPE
aapatvani-5eehpwheiq-el.a.run.app

01 What AapatVani is

AapatVani is a disaster-information system for the hill districts of Nepal. It takes the fragments a community sends during a monsoon event — a Nepali text message, a photograph, a forwarded message, a voice note from someone who saw it from a bus window — and turns them into one structured incident record a responder can act on.

The chain is the product: **community evidence → incident intelligence → potential impact analysis**, with provenance at every link. Most tools in this space stop at the second step, or at the first. A pin on a map tells a district officer that something happened somewhere. It does not tell them that four villages have just lost every mapped road route to a hospital whose nearest alternative is thirty kilometres away.

The one-sentence version

> During an event a district does not suffer from too few reports — it suffers from too many it cannot verify. AapatVani takes what the community already sends and turns it into one incident, with the copies and contradictions exposed and the road-access consequence computed.

What makes it different

ORDINARY TOOLS	WHAT AAPATVANI DOES INSTEAD
Crisis-mapping platforms collect reports and place pins, fast.	Reasons *over* the reports: groups them, excludes copies, flags contradictions, then computes what the blockage means for road access to services.
Hazard and satellite models predict flood extent, runout, damage.	Deliberately predicts **no** hazard. It answers a different, answerable question: what does what communities report imply for service access?
Government portals (Nepal's BIPAD and similar) are the official record.	Built to work *alongside* that world: community evidence, provenance-labelled, which a reviewer has confirmed before it is worth anything.
LLM summarisation reads many reports and writes fluent prose.	The summary is falsifiable. Every model output is schema-validated, citation-checked, and *downgradeable* when it cannot be supported.

Three properties enforced in code, not policy

1. **Provenance is mechanical.** The excerpt check, the alias map, the coordinate rule and the downgrade path are all tested. A claim the model cannot support is not softened — it is relabelled, and the label is visible on screen.
2. **Copies are a first-class failure mode.** Counting *independent witnesses* rather than *reports* is a small idea with a large effect on whether a responder trusts what they are reading. One forwarded message that looks like fifty confirmations is the most common way community-sourced information misleads.
3. **The system abstains.** A report with no usable location is never guessed at. A place outside the dataset is reported as *unchecked*, never as safe.

02 The problem, and why Nepal

Nepal's monsoon runs roughly June to September. Every year it triggers thousands of landslides across the middle hills. The characteristic failure is not the landslide itself — it is that **a hillside road is the only road**. A single slip at one point on the Arniko Highway can sever the connection between a dozen villages and their nearest health facility, sometimes for days.

The information problem sits on top of the physical one. Reports arrive through whatever channel is nearest to hand: a phone call to a relative, a Facebook post, a Viber message forwarded four times, a photo with no place attached. A district officer then has to work out, quickly and under pressure, which of these describe the same event, which are copies, which contradict each other, and what it all means for the villages downstream.

Why Nepal specifically

CONDITION	WHY IT MAKES THIS THE RIGHT PLACE TO BUILD
Single-road dependency	Hill settlements are connected by one mapped road. This makes the road-access question unusually decisive — the answer is often binary, not incremental.
High mobile penetration, low friction to report	A web page works. Nobody needs to install an application for an event that happens twice a year, and no account is required.
Language and literacy	Nepali is the working language of the affected districts, and typing Devanagari on a phone is a real barrier. Voice notes are not a nicety here; they are the difference between a person reporting and not.
Existing institutional context	Nepal already has BIPAD, the national disaster portal. The gap is not an official record — it is *community* evidence that has been checked. Working alongside an existing system is a better problem than filling a vacuum.
Open map data quality	OpenStreetMap coverage in Nepal is good enough to route over, and honest enough that its gaps can be reported as gaps rather than hidden.
Transferability	Every mechanism here — the road graph, the gazetteer, the hazard list, the region label — is configuration, not code. The same system applies to any road-constrained, monsoon-affected region.

The demo area

The prototype covers the **Araniko Highway corridor** in Sindhupalchok district: Dolalghat–Chautara–Bahrabise–Tatopani–Kodari, inside Nepal's border. This is a real OpenStreetMap extract, not a toy graph, and it contains a genuine, verifiable consequence:

IF THIS STRETCH OF THE ARNIKO HIGHWAY IS BLOCKED	WHAT THE ENGINE COMPUTES
Between Larcha and Tatopani	Tatopani, Namtan and Kodari lose **every mapped road route** to a health facility. The nearest alternative is 23–30 km away.
Near Ghumthang	20 settlements' routes to care grow by 14–15 km.
Near Khukundol	4 settlements' routes grow by 12 km.

These numbers come from the shipped dataset and can be reproduced with `npm run bench` and the impact panel in the running app.

03 What happens when someone reports

This is the whole pipeline, in order. Each stage names what it produces and who or what decides.

1 · Submit	Reporter	A phone browser posts a form to `/api/reports`. The server checks origin, rate limit and size, validates the fields, stores media to private storage, hashes each file, reads EXIF, and writes the report with a reference code. **The report is saved before any model is called**, so a Gemini outage can never lose a submission.
2 · Extraction	Gemini, then code	Gemini reads the text, photos and voice note and returns structured JSON. The server then checks it independently: schema validation, alias mapping, **excerpt verification against the reporter's own words**, and gazetteer resolution for place names. Coordinates are never accepted from the model.
3 · Matching	Code, then Gemini	Candidate pairs are filtered by region, time and distance, with compatible hazards. Text embeddings give a cross-lingual similarity score. Copies are detected by photo hash, near-identical wording, and reporter token. Gemini then gives a pairwise verdict citing claims — but **a score never merges anything**.
4 · Review	A person	In the reviewer workspace the original sits beside what was read from it, with excerpts highlighted on hover. Confirm, correct, reject or ask for more — per claim, and per grouping. Every action writes to an append-only audit trail.
5 · Impact	Code	When a claim describes a road blockage, the engine snaps it to a road segment, removes that segment from the real graph, and re-runs multi-source shortest paths from every health facility. Each settlement is classified *lost*, *longer* or *unchanged*. Runs in ~11 ms.
6 · Alert	Gemini, then a person	A draft is built from **confirmed facts only**. Every sentence cites its sources; sentences with no valid citation, or a number not present in their sources, are removed and counted. A named reviewer approves it. **The app never broadcasts.**

What comes out the other end

> Not "forty reports". Instead: **one landslide at Larcha. Three independent witnesses, one forwarded copy excluded. One report disagrees about whether the road is open. If that segment is blocked, four settlements lose every mapped route to a health facility — the nearest was 21 to 30 km away.**

04 The trust model

This is the part that makes the rest usable. Every claim, finding and map feature carries an **origin** and a **review status**, and the interface never lets either be invisible.

ORIGIN	WHAT IT MEANS	WHAT IS STORED WITH IT
Reported	A community member said it.	The report ID and the verbatim excerpt, verified against the original.
AI interpretation	Gemini read it from text, a photo or audio.	Model ID, prompt version, and the cited excerpt or media item.
Computed	Deterministic code over our datasets.	Dataset version, parameters and inputs.
Reviewer	A person entered or corrected it.	User ID, timestamp and note.

Review status runs `unreviewed` → `confirmed` · `edited` · `rejected` · `needs_info`. The word **Confirmed** appears in exactly one situation: after a named person has acted. Everything else is awaiting review.

The three rules the model is not trusted with

1. **Gemini never sees or returns real record IDs.** It cites short aliases — `R1` for the report, `M1` for a photo, `A1` for a voice note — which the server maps back. An alias it does not recognise causes the claim to be *dropped*.
2. **Every quoted excerpt must appear in the original text, character for character.** If it does not, the claim is downgraded to "AI interpretation, excerpt not verified" and flagged, rather than quietly kept.
3. **Coordinates never come from the model.** They come only from a GPS fix, a dropped pin, or a gazetteer match against real map data. A report with no usable location is deliberately never distance-compared, rather than guessed at.

> **Privacy.** Photographs sent to the model have their EXIF block stripped first — the reporter agreed to a machine reading their picture of a blocked road, not to their coordinates going to a third party with it. The stored original is untouched, so a reviewer still sees the true file. No account, name or phone number is ever required of a reporter.

05 Architecture: which service does what

One deployable. A modular Next.js 16 application on Cloud Run — no microservices, no multi-agent orchestration.



Service by service

SERVICE	ROLE	WHY THIS ONE
Cloud Run	The application	One container, scales to zero, no servers to manage. The hackathon requires deployment on Cloud Run, GCP or Firebase.
Gemini (via the Gemini API)	Reading reports, comparing them, drafting	Multimodal — text, image and audio in one model — with structured JSON output and multilingual embeddings. Nepali is first-class.
Firestore	The record store	Append-only collections, scales without administration, and reachable *only* from the server through the Admin SDK.
Cloud Storage	Photos and voice notes	Private bucket. Media is streamed to signed-in reviewers through the app, never served directly.
Secret Manager	Session key and reviewer passcode	Never in plain environment variables. The runtime service account is granted access to those two secrets only.
Cloud Build + Artifact Registry	Image build and storage	Builds the container from the Dockerfile. No local Docker required.
OpenStreetMap	The geography	Pre-processed offline into a versioned JSON graph that ships with the app, so the analysis is deterministic and needs no network at runtime.

Design principles that shaped the build

- **Swappable backends.** `DATA_BACKEND=local|firestore`, `MEDIA_BACKEND`, `AUTH_MODE=passcode|firebase`. The whole application runs on a laptop with no cloud credentials and moves to Google Cloud with environment variables only. This is why it could be built and tested for a week before the first credential existed.
- **Server-only data access.** Rules deny all client access to Firestore and Storage. Every read is a session-checked route handler. Access control lives in one place.
- **Save first, analyse second.** A report is stored and given a reference code before any model call.
- **Deterministic geography.** The same inputs always give the same road analysis. That is tested, and it is what makes the impact figures reproducible.

06 Running it

Locally, with no credentials at all

```
npm install
npm run seed        # the demo scenario: 6 synthetic reports over real road geography
npm run dev         # http://localhost:3000
```

Open `http://localhost:3000` for the public report form. Reviewers sign in at `/login` with the passcode `demo-reviewer` (local development only).

> **No credentials are needed for any of this.** Without a Gemini key the app still stores reports and marks them "awaiting a person". Review, grouping, contradiction detection and the road-impact analysis all work, because they are deterministic code rather than model calls.

The full command set

COMMAND	WHAT IT DOES
`npm run dev`	Development server on port 3000.
`npm run build`	Production build.
`npm test`	220 unit and end-to-end tests.
`npm run typecheck`	TypeScript, regenerating route types first.
`npm run lint`	ESLint.
`npm run seed`	Writes the demo scenario and runs matching over it.
`npm run seed:reset`	Clears the local store.
`npm run seed:reset:cloud`	Clears the deployed Firestore. Requires `--cloud` and refuses without it.
`npm run eval`	Measured results over the labelled scenarios.
`npm run eval -- --mode oracle`	The deterministic half, no model needed.
`npm run eval -- --mode gemini`	Real extraction; needs credentials.
`npm run check:models`	Verifies model IDs, structured output and embeddings against the live key.
`npm run test:multimodal`	Runs real extraction over a photo and a voice note.
`npm run bench`	Times the road analysis on the real corridor graph.
`npm run data:fetch`	Re-pulls the OSM extract from Overpass.

Configuration

Everything is environment-driven and documented in `.env.example`. With nothing set, the app uses a local file store, local media and passcode sign-in. The switch to Google Cloud is a set of variables:

VARIABLE	EFFECT
`DATA_BACKEND`	`local` (JSON files) · `firestore`
`MEDIA_BACKEND`	`local` (directory) · `gcs`
`AUTH_MODE`	`passcode` (shared, signed cookie) · `firebase`
`STORAGE_BUCKET`	The private media bucket
`GEMINI_API_KEY`	Developer API key, for local work
`GEMINI_MODEL_*`	Per-stage model overrides (extraction, reasoning, drafting, fallback, embedding)
`GEMINI_DAILY_BUDGET`	Hard daily request ceiling, enforced by the quota guard

The deployed prototype

```
https://aapatvani-5eehpwheiq-el.a.run.app

Project    aapatvani-511617   ·   Region  asia-south1 (Mumbai)
Runtime    Cloud Run, scale to zero, max 2 instances
Data       Firestore + private Cloud Storage bucket
Auth       passcode, session key in Secret Manager
```

07 What each user experiences

A community reporter

They open a web page on a phone. There is no application to install, no account, and no name to give. The interface is in Nepali by default, with English one tap away.

- They describe what happened — **by typing, by taking a photo, or by speaking**.
- They pick a hazard from the region's own list, with a plain-language description beside each name, because "landslide" asks a frightened person to already know the right word.
- They give a location three ways: a GPS fix, a pin dropped on a map, or simply **the name of the place**.
- They say roughly when it happened.

They receive a reference code and a receipt page showing **what the system understood** — their own words, the places recognised, the time interpreted. That page is theirs alone, gated on a token their browser holds; it deliberately shows no reviewer notes, no grouping, and no incident data.

If their place name is not in our map data, the app says so plainly rather than showing nothing: "*named by the reporter, not on our map*", with the consequence stated — that report was not compared with others by distance.

A reviewer

They sign in and land on a dashboard answering four questions in the order a shift runs: how much is waiting, where things are, which incidents exist, and what arrived recently.

- **The review queue** and **report workspace**: the original beside what was read from it. Hovering a claim highlights its excerpt in the Nepali text.
- **The incident workspace**: membership, scored match suggestions with a visible feature breakdown, and contradictions — each with confirm / correct / reject per claim.
- **The impact panel**: run the road analysis, see the assumed blocked segment, the affected settlements and the facilities whose catchment changes, each with its own caveats beside it.
- **The alert panel**: a draft built only from confirmed claims, every sentence citing its source. Approving it records a name and states that nothing was sent.
- **The timeline**: the evidence chain drawn on a horizontal axis — above the line what the community reported, below it what the system and reviewers did about it.

What the interface never does

- It never shows a colour without an icon and a word. Every state is legible without relying on colour vision.
- It never says "safe". Being outside the analysis area is stated as *unchecked*, and every impact panel carries the line "*Places outside this analysis are not necessarily safe*".
- It never hides a decision behind a score. Confirm, dismiss, merge and separate are all explicit acts by a named person.

08 What exists today

220
automated tests passing

11,340
road junctions in the shipped graph

36
health facilities, 4 of them hospitals

187
settlements analysed per run

~11 ms
road-access analysis, warm average

2,525 km
of mapped road in the extract

Built and verified

AREA	STATE
Public side — landing page, report form, receipt	Built. Nepali and English, photo, voice, three location modes.
Reviewer side — dashboard, queue, report and incident workspaces, settings	Built and exercised end to end on the seeded scenario.
Matching, copies, contradictions	Built. Deterministic rules plus a model verdict that cites claims.
Road-impact analysis	Built. ~11 ms warm on the real corridor graph, benchmarked by `npm run bench`.
Alert drafting and approval	Built. Citation and number checks enforced server-side.
Evaluation harness	Built. Oracle mode runs today; Gemini mode needs credentials.
Deployment	Live on Cloud Run, verified in a browser and by API.

Measured results

Oracle mode feeds the answer key in as if extraction had produced it, so it measures the deterministic half of the pipeline exactly, with no model involved:

METRIC	RESULT
Grouping precision / recall	100% / 100%
False merges	0
Copy detection precision / recall	100% / 100%
Contradiction recall	2 / 2
Correct abstentions	2 / 2
Impact scenarios correct	1 / 1

Read this honestly. These are five hand-written scenarios over real road geography. They show the pipeline behaves as designed on cases we chose. They are **not** a measure of real-world effectiveness, and the Gemini-mode run — the only one that can say anything about *reading reports* — has not yet been published. No claim beyond designed behaviour should be read from the table above.

09 What it deliberately does not do

Stating these is part of the design, not a disclaimer bolted on. A tool that is honest about its edges is more useful than one that is confidently wrong at them.

IT DOES NOT	BECAUSE
Predict hazards, runout, flood extent, casualties or damage	Those are physical models needing data we do not have. It answers the service-access question instead, which is answerable from what communities send.
Broadcast anything	Every alert is a draft a person approves. The application has no sending capability at all.
Merge reports automatically	A similarity score is evidence for a reviewer, never a decision.
Guess a location	A report with no GPS, pin or gazetteer match is left unplaced and never distance-compared.
Model multiple simultaneous blockages	The analysis assumes one segment is blocked at a time. Real landslides often bury a longer stretch; this is a known limitation, not an oversight.
Treat a paraphrase as a copy	Copies are judged on identical media, near-identical wording, or the same device. A light rewrite escapes detection today.
Cover the whole country	The dataset is the Araniko corridor. Outside it the app declines to analyse and says so.
Replace official systems	BIPAD is the official record. This is community evidence that a reviewer has checked.

Known weaknesses, stated plainly

- **The evaluation set is small and hand-written.** Five scenarios designed by the team. The system has never met a genuinely messy real dataset where forwarding means a screenshot, a re-recording, or a translation chain.
- **One reviewer does not scale to five hundred reports.** The human-in-the-loop design is right for correctness and is the throughput bottleneck under a mass event.
- **Voice transcription is imperfect.** Claims are checked against the transcript, and the transcript can itself contain errors — the honest limit of verifying a citation against a model's own reading.
- **OSM coverage in rural Nepal is uneven.** The analysis reports weak map links as caveats and lists already-disconnected settlements as data gaps rather than as impact.
- **Model availability on a free tier is not guaranteed.** The app carries a fallback ladder and degrades to "awaiting a person" rather than losing a report, but the strongest models are not always reachable.

10 Scaling

The important property is this: **value does not scale with adoption.** Nobody needs fifty villagers to download anything. One honest observation — one person saying the road is blocked near Larcha — is enough to run the whole engine and produce a complete access analysis. This is not a social network that needs critical mass; it is a machine that needs one report and a road graph.

Technical scaling

LAYER	HOW IT SCALES
Application	Cloud Run scales horizontally on requests. Stateless by design; no session affinity, no local state that matters.
Data	Firestore scales automatically. Records are append-only where it matters (audit, extractions), which keeps writes cheap and history intact.
Media	Cloud Storage, no practical ceiling. Streamed through an authorising route rather than served publicly.
Geography	The expensive part is the *offline preparation*, not the runtime. The graph is pre-built and shipped; analysis is ~11 ms regardless of traffic.
Model cost	The dominant per-report cost, and it is bounded: one extraction plus one embedding per report, plus one comparison per candidate pair that survives filtering.

Adding a region

Everything geographic is configuration, not code:

1. Run `scripts/prepare-region.ts` against an Overpass query for the new area. It produces the road graph, assets and gazetteer as versioned JSON.
2. Add a region config in `src/config/regions/` — its name, time zone, hazard list with local names, and dataset reference.
3. The app picks it up. Matching, impact analysis, the gazetteer and every screen read from the config.

A Philippines configuration already exists in the codebase, deliberately marked "no geographic dataset loaded, not validated" — the region switch works, the geography does not. It is a placeholder that demonstrates the seam rather than a claim about that country.

What would genuinely need to change at national scale

- **The reviewer bottleneck.** Human confirmation is the point, but it must become a shift-based queue with assignment, not one person reading everything. Priority ordering by independent-source count and impact severity is the natural next step.
- **Multi-segment blockages.** Real events block long stretches and cut several roads at once. The engine would need to accept a set of blocked segments rather than one.

- **A country-scale road graph.** The current dataset is a corridor shipped in the image. Nationwide would mean a spatial store rather than a bundled JSON file.
- **Institutional integration.** The natural end state is feeding confirmed incidents into BIPAD and police/army channels, in their format, with the provenance attached.
- **Offline-first intake.** SMS, IVR and a store-and-forward mobile client for areas with no data coverage during an event.

Beyond Nepal

The mechanism generalises to any road-constrained, hazard-exposed region with usable map data: hill districts in India, landslide-prone areas of the Philippines, mountain corridors in Peru or Georgia. What changes is the dataset and the hazard list. What does not change is the chain — community evidence, incident intelligence, road-access impact, with provenance throughout — and that is the part worth carrying across borders.

11 Future enhancements

ENHANCEMENT	EFFORT	VALUE
Report back to the reporter	Small	The receipt page today says what the system understood. It could also say what happened next — your report was confirmed, the road is being cleared. This closes the loop that makes someone report again.
Multiple blocked segments	Medium	Removes the most significant modelling limitation. The graph engine already handles segment removal; it needs to accept a set.
Priority queue for reviewers	Medium	Orders work by independent sources and computed impact rather than arrival time. Directly addresses the scaling bottleneck.
Paraphrase-resistant copy detection	Medium	Current detection catches identical wording. Embedding-similarity thresholds with human confirmation would catch the translation-chain and light-rewrite cases that matter in practice.
Offline and low-bandwidth intake	Large	SMS and IVR channels, plus a store-and-forward client. During a real event, coverage is exactly what fails.
BIPAD integration	Large	Pushes confirmed incidents into the official record rather than sitting beside it. Requires institutional agreement more than code.
Historical calibration	Large	Replay past monsoon seasons against recorded events to measure real recall and precision, replacing the hand-written evaluation set with something that can fail honestly.
Seasonal and weather context	Medium	Rainfall and soil-moisture context to rank which reported slips are most likely to worsen — stated as context, never as prediction.

The one we would do first. Reporting back to the reporter. It is the smallest change on this list and the only one that affects whether anyone uses the system a second time. Every other enhancement makes the machine better at its job; this one makes it worth someone's effort to feed.

12 Reference

Data sources and licences

SOURCE	USE	LICENCE
OpenStreetMap via the Overpass API	Road network, health facilities, schools and settlements for the Araniko corridor	ODbL 1.0. Attribution shown on every map.
OpenStreetMap standard tiles	Basemap in the interface	© OpenStreetMap contributors. Light demo use; the provider is switchable.

Extract snapshot **2026-10-11**: 11,340 nodes, 11,613 edges, 2,525 km of road, 36 health facilities (4 hospitals), 408 schools, 187 settlements, 622 gazetteer entries.

> **The reports are synthetic.** They are written by the team and labelled `synthetic` everywhere they appear. No real report and no real person's data is in this project. The geography, by contrast, is entirely real — real roads, real facilities, real distances.

Models

USE	MODEL
Extraction (text, photo, audio), comparison, contradictions, drafting	`gemini-3.8-flash`
Fallback when the primary is unavailable or rate-limited	`gemini-3.5-flash-lite`
Text embeddings for cross-lingual matching	`gemini-embedding-001`, 768 dimensions
Optional, hardest cases	`gemini-3.1-pro-preview`

All model IDs live in one file and are overridable by environment variable. The application records the model that actually answered on every extraction, alongside the prompt version and the raw output, so any result can be traced to what produced it.

Repository layout

```
project.md              the living tracker: status, decisions, progress log
docs/product-vision.md  the original brief
docs/deploy.md          Cloud Run deployment, step by step
src/app/                pages and API route handlers
src/components/         UI, grouped by area
src/config/             regions and model IDs, in one place each
src/lib/domain/         the Zod schemas — the shared contract
src/lib/ai/             Gemini calls, prompts, schemas, quota guard
src/lib/evidence/       aliases, excerpt verification, metadata stripping
src/lib/matching/       scoring, copy detection, contradictions
src/lib/geo/            gazetteer, road graph, shortest paths
src/lib/impact/         versioned road-access analysis
src/lib/alerts/         cited drafts and approval
data/regions/           the versioned OSM extract
data/eval/              saved evaluation results
scripts/                prepare-region, check-models, seed, eval, bench
tests/                  Vitest unit and end-to-end tests
```

Deliverables

REQUIREMENT	WHERE
Working prototype, deployed	`https://aapatvani-5eehpwheiq-el.a.run.app`
Source code and documentation, in English	This repository
Vision deck (PDF)	Submitted separately
Demonstration video, ~3 minutes	Public link, in the submission
Category	Social Good

AapatVani AI · आपतवाणी · Google Cloud AI Builder Cup 2026 · Theme: Sustainability & Social Impact · Category: Social Good. Built on Next.js 16, Gemini, Firestore, Cloud Storage and Cloud Run, over OpenStreetMap data © OpenStreetMap contributors (ODbL).